

## 查找排序算法

### 查找

#### 顺序查找（基本查找）

##### 数据结构

- 数组或链表

##### 算法思想

- 从数据结构线的一端开始，顺序扫描，依次将遍历到的结点与要查找的值相比较，若相等则表示查找成功；若遍历结束仍未找到相同的，表示查找失败。

#### 二分查找（折半查找）

##### 数据结构

- 元素必须是有序的，如果是无序的则要先进行排序操作

##### 算法思想

- 用给定值k先与中间结点的关键字比较，中间结点把线形表分成两个子表，若相等则查找成功；若不相等，再根据k与该中间结点关键字的比较结果确定下一步查找哪个子表，这样递归进行，直到查找到或查找结束发现表中没有这样的结点。

#### 插值查找

##### 数据结构

- 元素必须是有序的，如果是无序的则要先进行排序操作

##### 算法思想

- 基于二分查找算法，将查找点的选择改进为自适应选择，可以提高查找效率。当然，插值查找也属于有序查找。

#### 斐波那契查找

##### 数据结构

- 元素必须是有序的，如果是无序的则要先进行排序操作

##### 算法思想

- 二分查找的一种提升算法，通过运用黄金比例的概念在数列中选择查找点进行查找，提高查找效率

#### 二叉树查找算法

##### 数据结构

- 二叉查找树，2-3查找树，红黑树

##### 算法思想

- 先对待查找的数据进行生成树，确保树的左分支的值小于右分支的值，然后在就行和每个节点的父节点比较大小，查找最适合的范围。

#### 分块查找（索引顺序查找）

##### 数据结构

- 由“分块有序”的线性表和索引表组成

1) “分块有序”的线性表

表R[1..n]均分为b块，前b-1块中结点个数s=n/b，第b块的结点数小于等于s；每一块中的关键字不一定有序，但前一块中的最大关键字必须小于后一块中的最小关键字，即表是“分块有序”的。

2) 索引表

抽取各块中的最大关键字及其起始位置构成一个索引表ID[l..b]，即：ID[i](1≤i≤b)中存放第i块的最大关键字及该块在表R中的起始位置。由于表R是分块有序的，所以索引表是一个递增有序表。

##### 算法思想

- (1) 首先查找索引表

索引表是有序表，可采用二分查找或顺序查找，以确定待查的结点在哪一块

(2) 然后在已确定的块中进行顺序查找

由于块内无序，只能用顺序查找。

#### 哈希查找（散列查找）

##### 数据结构

- 哈希表

##### 算法思想

- 将元素的关键字通过哈希函数转换为唯一的地址，然后将元素存储在该地址中。这样，当需要查找某个元素时，只需计算其哈希值并访问对应的地址即可。

| 查找方法   | 时间复杂度       | 空间复杂度  | 稳定性 | 适用场景         | 优缺点          |
|--------|-------------|--------|-----|--------------|--------------|
| 顺序查找   | $O(n)$      | $O(1)$ | 稳定  | 数据量小，无序      | 简单，但效率低      |
| 二分查找   | $O(\log n)$ | $O(1)$ | 不稳定 | 数据量大，有序      | 效率高，但要求有序    |
| 插值查找   | $O(\log n)$ | $O(1)$ | 不稳定 | 数据量大，有序，分布均匀 | 比二分查找更快      |
| 斐波那契查找 | $O(\log n)$ | $O(1)$ | 不稳定 | 数据量大，有序      | 比二分查找更快      |
| 二叉树查找  | $O(\log n)$ | $O(n)$ | 不稳定 | 数据量大，有序      | 效率高，但空间复杂度高  |
| 哈希查找   | $O(1)$      | $O(n)$ | 不稳定 | 数据量大，无序      | 效率最高，但空间复杂度高 |

### 排序

#### 冒泡排序

| 排序方法 | 时间复杂度         | 空间复杂度       | 稳定性 | 适用场景    | 优缺点         |
|------|---------------|-------------|-----|---------|-------------|
| 冒泡排序 | $O(n^2)$      | $O(1)$      | 稳定  | 数据量小，无序 | 简单，但效率低     |
| 快速排序 | $O(n \log n)$ | $O(\log n)$ | 不稳定 | 数据量大，无序 | 效率高，但空间复杂度高 |
| 堆排序  | $O(n \log n)$ | $O(1)$      | 不稳定 | 数据量大，无序 | 效率高，但空间复杂度高 |
| 归并排序 | $O(n \log n)$ | $O(n)$      | 稳定  | 数据量大，无序 | 效率高，但空间复杂度高 |
| 基数排序 | $O(n)$        | $O(n)$      | 稳定  | 数据量大，有序 | 效率高，但空间复杂度高 |

#### 内部排序

指所有的数据已经读入内存，在内存中进行排序的算法。排序过程中不需要对磁盘进行读写。同时，内排序也一般假定所有用到的辅助空间也可以直接存在于内存中

#### 交换排序

##### 冒泡（Bubble Sort）

###### 时间复杂度

- 平均： $O(n^2)$ ；最好： $O(n)$ ；最坏： $O(n^2)$

###### 空间复杂度

- $O(1)$

###### 算法思想

- 重复的遍历（走过）待排序的一组数字（通常是列表形式），依次比较两个相邻的元素（数字），若它们的顺序错误则将它们调换一下位置，直至没有元素再需要交换为止。

##### 快速（Quick Sort）

###### 时间复杂度

- 平均： $O(n \log n)$ ；最好： $O(n \log n)$ ；最坏： $O(n^2)$

###### 空间复杂度

- 主要在递归调用时用到；平均： $O(\log n)$ ；最好： $O(\log n)$ ；最坏： $O(n)$

###### 算法思想

- 通过一趟排序将待排序列表分割成独立的两部分，其中一部分的所有元素都比另一部分小，然后再按此方法将独立的两部分分别继续重复进行此操作，这个过程我们可以通过递归实现，从而达到最终将整个列表排序的目的。

#### 选择排序

##### 直接选择（Straight Select Sort）

###### 时间复杂度

- 平均： $O(n^2)$ ；最好： $O(n^2)$ ；最坏： $O(n^2)$

###### 空间复杂度

- $O(1)$

###### 算法思想

- 先在待排序列表中找到最小（大）的元素，把它放在起始位置作为已排序序列；然后，再从剩余待排序序列中找到最小（大）的元素放在已排序序列的末尾，以此类推，直至完毕。

##### 堆排序（Heap Sort）

堆的结构相当于一个完全二叉树，最大堆满足下面的性质：父结点的值总大于它的孩子结点的值

###### 时间复杂度

- 平均： $O(n \log n)$ ；最好： $O(n \log n)$ ；最坏： $O(n \log n)$

###### 空间复杂度

- $O(1)$

###### 算法思想

- 将无序序列构建成一个堆，根据升序降序需求选择大顶堆或小顶堆；
- 将堆顶元素与末尾元素交换，将最大元素“沉”到数组末端；
- 重新调整结构，使其满足堆定义，然后继续交换堆顶元素与当前末尾元素，反复执行调整+交换步骤，直到整个序列有序。

#### 插入排序

##### 直接插入(Straight Insertion Sort)

###### 时间复杂度

- 平均： $O(n^2)$ ；最好： $O(n)$ ；最坏： $O(n^2)$

###### 空间复杂度

- $O(1)$

###### 算法思想

- 对于未排序元素，在已排序序列中从后向前扫描，找到相应位置把它插入进去；在从后向前扫描过程中，需要反复把已排序元素逐步向后挪位，为新元素提供插入空间。

##### 希尔排序(Shell Sort)

###### 时间复杂度

- 平均： $O(n \log n - n^2)$ ；最好： $O(n^{1.3})$ ；最坏： $O(n^2)$

###### 空间复杂度

- $O(1)$

###### 算法思想

- 将待排序列表按下标的一定增量分组（比如增量为2时，下标1, 3, 5, 7为一组，下标2, 4, 6, 8为另一组），各组内进行直接插入排序；随着增量的越来越小，每组所包含的数字序列越来越多，当增量减至1时，整个序列被分成一个组，排序就完成了。

#### 归并排序

##### 时间复杂度

- 平均： $O(n \log n)$ ；最好： $O(n \log n)$ ；最坏： $O(n \log n)$

##### 空间复杂度

- $O(n)$

##### 算法思想

- 递归的将两个已排序的序列合并成一个序列。
  - 申请空间，其大小为两个已经排序序列之和，该空间用来存放合并后的序列
  - 设定两个指针，最初位置分别为两个已经排序序列的起始位置
  - 比较两个指针所指向的元素，选择相对小的元素放入到合并空间，并移动指针到下一位置

#### 外部排序

内存中无法保存全部数据，需要进行磁盘访问，每次读入部分数据到内存进行排序

#### 计数排序（Counting Sort）

当输入的元素是 n 个 0到 k 之间的整数时

##### 时间复杂度

- $O(n+k)$

##### 空间复杂度

- $O(n+k)$

##### 算法思想

- 对于给定的输入序列中的每一个元素x，确定该序列中值小于x的元素的个数（此处并非比较各元素的大小，而是通过对元素值的计数和计数值的累加来确定）。一旦有了这个信息，就可以将x直接存放到最终的输出序列的正确位置上。
  - 统计列表L中每个值L[i]出现的次数，存入count[L[i]]
  - 从前向后，使列表count中的每个值等于其与前一项相加，这样列表count[L[i]]就变成了代表列表L中小于等于L[i]的元素个数
  - 反向填充目标列表target：将列表元素L[i]放在列表target的第count[L[i]]个位置（下标为count[L[i]] - 1），每放一个元素就将count[L[i]]递减

#### 桶排序（箱排序，Bucket Sort）

##### 时间复杂度

- 平均： $O(n)$ ；最好： $O(n)$ ；最坏： $O(n \log n)$ 或 $O(n^2)$

##### 空间复杂度

- $O(n + \text{bucket\_num})$

##### 算法思想

- 是计数排序的升级版，利用了函数的映射关系将数据分到有限数量的桶里，每个桶再分别进行排序（使用别的排序方法或者递归的继续使用桶方法排序）
  - 设置定量表（数组）当作空桶
  - 遍历待排序序列，将数据一个一个放到对应的桶里面
  - 对每个非空桶进行排序
  - 把排好序的非空桶里的序列拼接在一起

#### 基数排序（Radix Sort）

##### 时间复杂度

- 平均： $O(n * \text{bucket\_num})$ ；最好： $O(n * \text{bucket\_num})$ ；最坏： $O(n * \text{bucket\_num})$

##### 空间复杂度

- $O(n * \text{bucket\_num})$

##### 算法思想

- 将所有待比较正整数统一为同样的数位长度，数位较短的数前面补零。然后，从最低位开始进行基数为10的计数排序，一直到最高位计数排序完后，数列就变成一个有序序列
  - 确定位数基数
  - 从低位到高位，每位看作一个桶，每个桶内进行计数排序
  - 最高位桶排序完成后，整个排序便完成了